

Day04 Part D 学习文档 v1.0 : Runtime 如何思考 (How Runtime Thinks) - Context Window Management

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part D 正式学习文档。

Part A 建立了“Prompt 不是 Context”的世界观；Part B 解释 Runtime State 是什么以及它如何创建、演化、持久化和恢复；Part C 解释 Context Builder 如何把 Runtime State 投影成 Provider Messages。Part D 继续回答一个工业级问题：当 Context Window 有限时，Runtime 如何决定保留什么、压缩什么、淘汰什么，以及如何让 Agent 长时间连续运行。

与上一节的联系

Part C 的核心结论是：

Context Builder 是 Runtime 的认知调度中心，它把 Runtime State 投影成 LLM 当前可见的 Provider Messages。

但是 Part C 留下了一个更现实的问题：

Runtime State
=
历史对话
+ 长期 Memory
+ 当前 Goal
+ Workspace
+ Tool Result
+ Planner State
+ Checkpoint
+ Execution Metadata

这些信息可能远远超过模型的 Context Window。

所以 Part D 的核心问题是：

Runtime 如何在有限 Context Window 中，持续为 LLM 构造“足够相关、足够紧凑、足够连续”的当前世界？

Part D 的核心结论是：

Context Window Management 是 Agent Runtime 的认知资源管理系统。它负责在有限 token 空间中完成 Selection、Allocation、Ranking、Eviction、Compression 和 Recovery，使 LLM 每一轮都看到当前任务最需要的信息，而不是看到最多的信息。

可以这样串起来：

Part A:
Prompt 不是 Context

Part B-01:
Context 来自 Runtime State 的 Projection

Part B-02:
Runtime State 有完整生命周期

Part C:
Context Builder 把 Runtime State 投影成 Provider Messages

Part D:
Context Window Management 管理有限上下文空间下的工业级取舍

Part E:
Provider Adapter 补齐模型供应商适配层

Day05:
Tool Calling 进入执行系统

目录

Part D 总体目标

第一章：理解 Context Window (Context Window Fundamentals)

- 1.1 Context Window 到底是什么
- 1.2 Context Window、Memory 与 Conversation 的区别
- 1.3 为什么 1M Context 仍然不够
- 1.4 Agent 为什么必须管理 Context

第二章：Context Budget Management (上下文预算管理)

- 2.1 Token Budget 是什么
- 2.2 Context Allocation
- 2.3 Priority Ranking
- 2.4 Context Eviction

第三章：Context Compression (上下文压缩)

- 3.1 为什么需要 Context Compression
- 3.2 Summary Memory
- 3.3 Hierarchical Context
- 3.4 Long-running Agent 如何保持连续性

第四章：Context Engineering 实战设计

- 4.1 Context Builder 工业架构回顾
- 4.2 工业 Agent Context Pipeline
- 4.3 Mini Agent Context Manager 设计
- 4.4 Day04 总结与收尾

下一节学习计划

写书 TODO

写书素材

本章思考题

Part D 总体目标

Context Window Management 解决的不是“怎么把更多文本塞给模型”，而是：

如何在有限窗口中管理 Agent 的当前认知。

一个工业级 Agent Runtime 必须回答四类问题：

当前模型调用最多能看多少 token ？

这些 token 应该分配给哪些信息区域 ？

当候选信息超过预算时，应该删掉什么、压缩什么、外部化什么？

当 Agent 运行很久后，如何保持目标、决策、约束和状态连续？

本 Part 把学习内容拆成四大章、十六小节：

第一章：Context Window Fundamentals

- └─ 1.1 Context Window 到底是什么
- └─ 1.2 Context Window != Memory != Conversation
- └─ 1.3 为什么 1M Context 仍然不够
- └─ 1.4 Agent 为什么必须管理 Context

第二章：Context Budget Management

- └─ 2.1 Token Budget 是什么
- └─ 2.2 Context Allocation
- └─ 2.3 Priority Ranking
- └─ 2.4 Context Eviction

第三章：Context Compression

- └─ 3.1 为什么需要 Context Compression
- └─ 3.2 Summary Memory
- └─ 3.3 Hierarchical Context
- └─ 3.4 Long-running Agent 如何保持连续性

第四章：Context Engineering 实战设计

- └─ 4.1 Context Builder 工业架构回顾
- └─ 4.2 工业 Agent Context Pipeline
- └─ 4.3 Mini Agent Context Manager 设计
- └─ 4.4 Day04 总结

第一章：理解 Context Window (Context Window Fundamentals)

1.1 Context Window 到底是什么

Context Window 是一次模型调用中，模型能够处理的最大 token 空间。

它通常包括：

- system message ；
- developer message ；
- user message ；
- assistant history ；
- tool definition ；
- tool result ；
- memory ；
- workspace 片段 ；
- 当前计划 ；
- 输出预留空间。

可以简单表示为：

Context Window
=

Input Tokens
+
Output Tokens

所以 Context Window 不是“只给输入用的空间”。如果把全部窗口都塞给输入，模型就没有足够空间生成输出。

1.1.1 Token 不是字符

Token 是模型处理文本的基本单位，不等于字符，也不等于单词。

同样长度的文本，在不同语言、不同符号密度、不同代码结构下，token 消耗会不同。

Agent 特别容易消耗 token，因为它不只处理用户一句话，还会携带：

- 系统规则；
- 工具定义；
- 历史对话；
- 文件内容；
- 工具结果；
- 当前计划；
- 错误日志；
- Memory；
- 输出格式要求。

1.1.2 Context Window 的本质

Context Window 是模型当前看到的世界。

LLM 本身没有 Runtime State，它只能基于当前窗口中的内容推理。窗口之外的信息，即使存在于数据库、文件系统或历史对话中，模型在这一轮也看不到。

因此：

Runtime State 是 Runtime 的完整世界。
Context Window 是 LLM 本轮可见的世界。

Agent Runtime 的关键能力，不是让模型看到一切，而是让模型每一轮看到最该看到的东西。

1.2 Context Window、Memory 与 Conversation 的区别

很多 Agent 初学者会把三者混在一起：

Context = Memory = Conversation

这是一个很危险的简化。

1.2.1 Conversation

Conversation 是用户与 Agent 之间发生过什么。

它的特点是：

- 时间顺序强；
- 更像事件日志；
- 记录用户、assistant、tool 之间的交互；
- 可以很长；

- 不一定全部进入 Context。

Conversation 的价值是可追溯。

1.2.2 Memory

Memory 是 Runtime 从经历中沉淀出的长期可复用信息。

它可能来自 Conversation，也可能来自用户偏好、项目事实、长期经验或显式保存的信息。

Memory 的特点是：

- 跨 Session 保留；
- 语义密度更高；
- 需要检索、排序和治理；
- 不等于完整聊天记录；
- 也不是每一条都应该进入 Context。

Memory 的价值是长期复用。

1.2.3 Context Window

Context Window 是本轮 LLM 调用实际看到的信息集合。

它的特点是：

- 生命周期很短；
- 受 token budget 限制；
- 面向当前 Goal 动态生成；
- 由 Context Builder 从 Runtime State 中投影而来；
- 不是持久化的知识库。

Context Window 的价值是当前推理。

三者关系可以表示为：

Conversation 记录发生过什么
Memory 记录值得长期复用什么
Context 决定本轮 LLM 应该看到什么

前端类比：

Redux Store -> Runtime State
Selector -> Context Builder
Rendered DOM -> Context Window
Component Render -> LLM 本轮推理

1.3 为什么 1M Context 仍然不够

更大的 Context Window 很有价值，但它不等于无限上下文，也不等于更强 Agent。

1.3.1 成本问题

更多 token 意味着更高的推理成本。

工业 Agent 需要在大量用户、多轮调用、工具循环和长期任务中运行。每次都塞满 Context Window，会让成本快速失控。

1.3.2 延迟问题

更长输入通常会带来更长处理时间。

Coding Agent、客服 Agent、知识库 Agent

这类交互式系统都需要及时反馈。如果每一轮都处理超大上下文，用户体验和执行效率都会下降。

1.3.3 噪声问题

Context 大了以后，进入窗口的信息也更容易变杂。

LLM 不是看得越多就越聪明。无关信息会制造噪声，让模型在推理时分散注意力，甚至把旧约束、旧决策或不相关事实误当成当前依据。

1.3.4 Attention 稀释问题

即使模型能处理很长上下文，也不代表它会同等质量地利用所有信息。

在长上下文中，关键信息可能被埋在大量低价值信息里，导致模型：

- 忽略关键约束；
- 引用过期信息；
- 混淆多个任务；
- 生成看似合理但上下文依据不足的结果。

所以工业级目标不是：

把所有信息都放进去。

而是：

让有限窗口保持高信息密度、高相关性和高连续性。

1.4 Agent 为什么必须管理 Context

普通聊天可以依赖“最近几轮对话 + 当前用户输入”。

Agent 不行。

Agent 通常要执行多轮任务：

```
User Goal
-> Plan
-> Tool Call
-> Tool Result
-> State Update
-> New Context
-> LLM Decision
-> More Tool Calls
```

每一轮需要的 Context 可能都不一样。

例如修复一个登录 bug：

第一轮可能需要：

- 用户目标；
- 报错信息；
- 相关文件路径；
- 最近对话。

第二轮可能需要：

- 已读过的代码摘要；
- 测试失败结果；
- 修改计划；
- 认证模块调用链。

第三轮可能需要：

- diff；
- 测试结果；
- 剩余问题；
- 用户要求。

这说明 Context 不是静态 Prompt，而是随着 Agent Loop 动态变化的认知视图。

Context Management 的四个核心动作是：

- Selection：选择哪些信息进入窗口；
- Allocation：给不同信息区域分配 token；
- Compression：把长信息压缩成高密度表达；
- Recovery：当溢出或信息丢失时恢复连续性。

第二章：Context Budget Management (上下文预算管理)

2.1 Token Budget 是什么

Token Budget 是 Runtime 对一次 LLM 调用的上下文空间分配策略。

错误模型是：

```
max_context = input
```

正确模型是：

```
max_context
=
system budget
+ goal budget
+ memory budget
+ conversation budget
+ workspace budget
+ tool result budget
+ output reserve
```

一个示例：

Total Context Window: 128k

```
System Policy: 4k
Current Goal: 2k
Recent Messages: 12k
Memory: 8k
Workspace: 64k
Tool Results: 18k
Plan State: 4k
Output Reserve: 16k
```

Output Reserve 非常关键。

如果 Runtime 把窗口全部用于输入，LLM 就没有足够空间完成回答、代码生成、工具调用参数生成或复杂推理。

因此 Token Budget Manager 需要同时管理：

- 输入预算；
- 输出预留；
- 不同信息源之间的分配；
- 溢出时的降级策略。

2.2 Context Allocation

Context Allocation 是把有限 token 分配给不同上下文区域。

它不是固定比例，而是由当前任务动态决定。

2.2.1 常见区域

一个 Agent Context 通常会分成：

- System Budget：系统规则和不可破坏的约束；
- Goal Budget：用户当前目标和明确要求；
- Memory Budget：长期偏好、项目事实、历史决策；
- Conversation Budget：最近对话和关键上下文；
- Workspace Budget：相关文件、diff、代码片段、文档；
- Tool Result Budget：命令输出、搜索结果、测试结果；
- Output Reserve：模型输出空间。

2.2.2 Allocation 如何动态识别任务

Context Allocation 依赖 Task Understanding Layer。

这个层会根据用户请求、历史状态、工具结果和工作区信号识别任务类型。

常见方式包括：

- LLM 判断：让模型基于请求识别任务类型；
- 规则系统：根据关键词、命令、文件路径、操作类型判断；
- Retrieval 信号：通过关键词、embedding、symbol index、AST index 找相关内容；
- 历史 State 推断：根据当前 plan step、上一步 tool result、失败原因判断；
- 混合策略：先用规则做粗分类，再用模型或检索做细化。

例如：

Debug 任务：
更多预算给 error log、相关源码、最近测试结果。

Feature 任务：
更多预算给需求、相关模块、设计约束、已有接口。

Code Review：
更多预算给 diff、风险规则、历史约定、测试结果。

概念解释:

更多预算给用户问题、前置知识、教学结构。

所以 Context Allocation 本质上是 Context Scheduling。

2.3 Priority Ranking

Priority Ranking 决定候选信息进入 Context 的优先级。

它不是简单按时间排序。

工业 Context Ranking 通常考虑：

- Relevance：与当前 Goal 的相关性；
- Importance：对任务成功是否关键；
- Recency：是否足够新；
- Dependency：是否是后续推理依赖；
- User Constraint：是否来自用户显式要求；
- Trust：信息来源是否可靠；
- Cost：进入上下文的 token 成本；
- Recoverability：被移除后是否容易恢复。

一个简化打分函数可以写成：

```
score(item)
=
relevance * 0.35
+ importance * 0.25
+ recency * 0.15
+ dependency * 0.15
+ user_constraint * 0.10
- token_cost_penalty
```

Ranking 和 Retrieval 不同。

Retrieval 负责发现候选信息。

Ranking 负责决定候选信息的优先级。

Selection 负责决定最终进入 Context 的集合。

搜索引擎类比：

Retrieval:

先找出可能相关的网页。

Ranking:

把这些网页按价值排序。

Context Selection:

选择首页应该展示哪些结果。

Agent Runtime 中的 Context Builder 也在做类似事情，只是目标不是展示搜索结果，而是构造 LLM 的当前认知。

2.4 Context Eviction

Context Eviction 是在 Context 超过预算时，把一部分信息从当前窗口中移出。

它不是 Memory 删除。

它只表示：

这条信息本轮不进入 LLM 可见窗口。

信息仍然可以存在于：

- Conversation Store ；
- Memory Store ；
- Workspace ；
- Tool Result Store ；
- Checkpoint ；
- Summary ；
- 外部索引。

2.4.1 为什么不能简单 LRU

操作系统内存淘汰可以参考 LRU，但 Agent Context 不能简单照搬。

因为旧信息不一定不重要。

例如：

- 用户最开始给出的硬性约束；
- 项目的架构原则；
- 已经做出的关键技术决策；
- 当前任务的目标；
- 安全和权限规则；
- 长期偏好。

这些信息即使很早出现，也可能必须保留。

2.4.2 Eviction 的核心维度

工业 Eviction 通常考虑：

- Importance：是否关键；
- Relevance：是否与当前任务相关；
- Compression Ability：是否可以压缩；
- Recoverability：移出后是否能重新检索；
- Staleness：是否过期；
- Conflict Risk：是否与当前状态冲突。

2.4.3 四种 Eviction 策略

第一种是 Truncation。

直接截断低价值或过长内容。这是最简单但风险最高的方法。

第二种是 Summarization。

把长对话、长日志、长工具结果压缩成摘要。

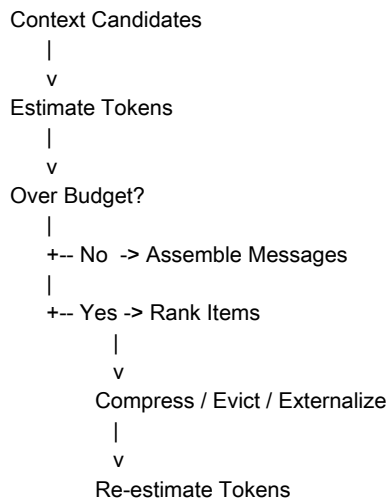
第三种是 Selective Removal。

只移除与当前任务无关的段落或项目。

第四种是 Externalize。

把完整信息移到外部存储，Context 中只保留引用、摘要和重新取回线索。

完整流程可以表示为：



第三章：Context Compression (上下文压缩)

3.1 为什么需要 Context Compression

Context Compression 是把低密度、长篇幅信息转换成高密度、可推理信息。

它不是简单删字，也不等于普通总结。

核心目标是保留：

- Goal ；
- Constraints ；
- Decisions ；
- Rationale ；
- Current State ；
- Open Questions ；
- Dependencies ；
- Next Actions。

更准确地说，Compression 要保留的是决策边界。

例如一个任务讨论了三个方案：

方案 A 被选择，因为它最小化改动，并兼容现有接口。
方案 B 被放弃，因为会破坏向后兼容。
方案 C 被放弃，因为需要额外基础设施。

压缩后不能只写：

讨论了三个方案，选择方案 A。

更好的压缩是：

已选择方案 A。原因是改动小且兼容现有接口。

不要采用方案 B，因为它破坏向后兼容。
暂不采用方案 C，因为它引入额外基础设施成本。

这里保留下来的不是文本表面，而是未来推理需要遵守的边界。

3.2 Summary Memory

Summary Memory 是最常见的 Context Compression 方法之一。

它把长对话或长执行过程压缩成可继续工作的状态摘要。

3.2.1 Summary Memory 保存什么

工业 Summary Memory 不应该只保存“聊了什么”，而应该保存：

- Goal：当前目标是什么；
- Completed Actions：已经完成了什么；
- Decisions：做过哪些关键决策；
- Constraints：必须遵守哪些约束；
- Current State：当前状态是什么；
- Open Questions：还有哪些未解决问题；
- Next Steps：下一步应该做什么；
- Risks：已有风险和注意事项。

3.2.2 Summary Memory 和 Conversation History

Conversation History 记录的是完整过程。

Summary Memory 记录的是可继续执行的状态。

Conversation:
事件日志。

Summary Memory:
状态摘要。

这和 Git Commit 有点像。

完整开发过程可能很长，但一次 commit message 应该保留：

- 做了什么；
- 为什么做；
- 影响范围；
- 后续注意点。

3.2.3 Summary Memory 的问题

Summary Memory 最大风险是 Information Loss。

压缩时如果漏掉约束、原因或未解决问题，后续 Agent 就可能出现状态断裂。

另一个风险是 Summary Accumulation。

摘要如果不断叠加，旧摘要可能变长、变旧、冲突或污染新上下文。

所以 Summary Memory 自身也需要治理：

- 定期重写；
- 合并重复；
- 标记过期；
- 保留来源；
- 支持展开原始 Conversation；
- 根据任务重新检索。

3.3 Hierarchical Context

Hierarchical Context 是把上下文组织成多层结构，而不是把所有信息平铺进窗口。

典型层次：

- Level 0: Current Goal
- Level 1: Current Step
- Level 2: Working Summary
- Level 3: Relevant Memory
- Level 4: Relevant Files / Tool Results
- Level 5: Full Conversation / Full Documents / Raw Logs

模型当前通常只看到上层高密度内容。

当需要细节时，再通过工具、检索或展开机制访问下层。

这就是 Progressive Disclosure。

3.3.1 为什么需要分层

分层可以解决三个问题：

- 降低 token 消耗；
- 保持长期一致性；
- 支持按需展开细节。

如果把所有信息平铺到 Context 中，模型会被迫同时处理大量不同粒度的信息。

分层之后，Runtime 可以先提供摘要、目标、约束和当前状态，再按需展开源材料。

3.3.2 和 RAG 的关系

RAG 更偏向“检索相关知识”。

Hierarchical Context 更偏向“组织 Agent 当前认知”。

它可以使用 RAG，但不等于 RAG。

一个好的 Agent Context 系统通常会同时具备：

- Retrieval；
- Ranking；
- Summary；
- Memory；
- Workspace Index；
- Hierarchical Organization；

- Context Assembly。

3.4 Long-running Agent 如何保持连续性

Long-running Agent 是可以跨越大量轮次、长时间运行、不断调用工具、不断更新状态的 Agent。

LLM 本身没有连续记忆。

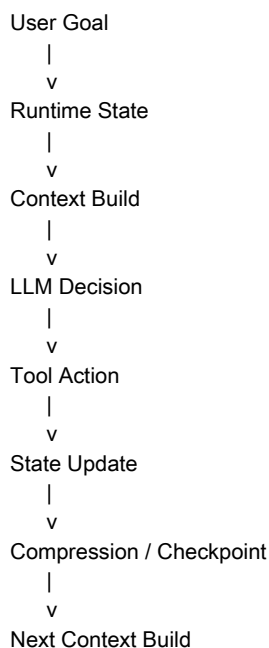
连续性来自 Runtime。

3.4.1 Long-running Agent 的核心状态

Runtime 需要维护：

- Goal State：用户目标和成功标准；
- Plan State：计划、阶段、当前步骤；
- Execution State：已执行动作、失败、重试、结果；
- Context Summary：当前可继续工作的摘要；
- Tool State：工具调用状态、权限状态、外部资源状态；
- Checkpoint：可恢复的里程碑；
- Memory：长期可复用知识。

3.4.2 连续性的关键循环



每一轮 LLM 调用都不是从零开始，而是 Runtime 根据当前 State 重新构造上下文。

3.4.3 为什么 Plan 文档有价值但不等于 Memory

OpenSpec、Claude Code Plan 模式或项目执行文档可以帮助 Agent 保持任务结构。

但 Plan 文档不自动等于 Memory。

原因是：

- 文档可能只是当前任务 Artifact；
- 旧计划可能过期；

- 多个计划可能互相冲突；
- 文档需要被检索、压缩、整合和治理；
- Agent 需要知道什么时候相信它、什么时候更新它。

所以更准确的关系是：

```
Plan Document -> Candidate Knowledge
Candidate Knowledge -> Memory Consolidation
Memory Consolidation -> Managed Memory
Managed Memory -> Context Builder Projection
```

Artifact 只有被 Runtime 治理之后，才会真正变成可靠 Memory。

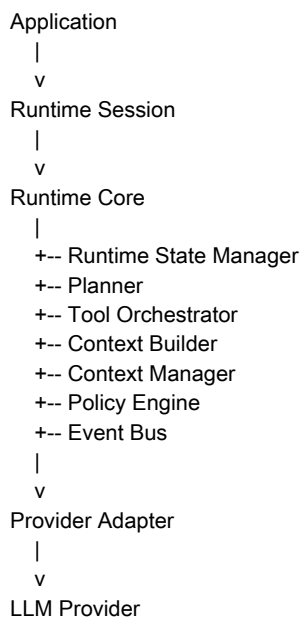
第四章：Context Engineering 实战设计

4.1 Context Builder 工业架构回顾

经过 Part C 和 Part D，Context Builder 不再是 Prompt Template。

它更像 Agent 的 Context Management System。

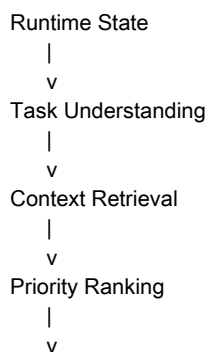
4.1.1 Context Builder 在 Runtime 中的位置

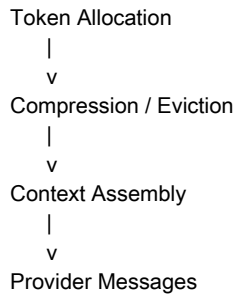


Context Builder 属于 Runtime Core，因为它决定 LLM 当前看见什么。

Provider Adapter 属于模型协议适配层，它决定统一消息如何转换成不同供应商的请求格式。

4.1.2 Context Builder 的工业流程





每一步都解决不同问题：

- Task Understanding：判断当前任务类型和目标；
- Retrieval：找到候选信息；
- Ranking：排序候选信息；
- Allocation：分配 token；
- Compression：提高信息密度；
- Eviction：移出低价值信息；
- Assembly：组装最终消息。

4.1.3 为什么 Context 顺序重要

Context 不只是信息集合，也是一种阅读路径。

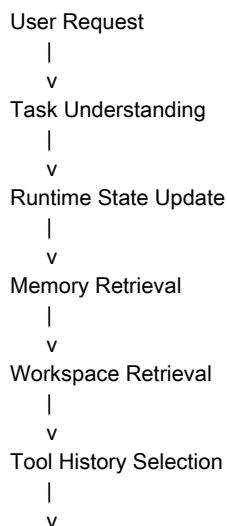
一般来说，模型更容易按照结构化顺序推理：

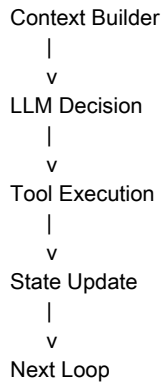
System Policy
Current Goal
Hard Constraints
Relevant State Summary
Recent User Request
Relevant Memory
Workspace Evidence
Tool Results
Output Requirements

如果顺序混乱，模型可能不知道哪些是规则、哪些是背景、哪些是当前任务、哪些是旧信息。

4.2 工业 Agent Context Pipeline

一个 OpenAI / Claude Code 类 Coding Agent 的 Context Pipeline 可以概括为：





Claude Code 类 Agent 效果好的原因，不是只因为模型强，而是因为它有更完整的运行时系统：

- Repository Understanding ;
- Workspace Retrieval ;
- Tool Loop ;
- Context Management ;
- Human-like Planning ;
- 文件、命令、diff、测试结果之间的状态连接；
- 长任务中的摘要和连续性管理。

更准确地说：

模型决定推理能力上限。

Runtime 决定 Agent 在真实世界中的表现。

Context Engineering 决定模型每一轮能否站在正确位置思考。

4.3 Mini Agent Context Manager 设计

一个最小可运行的 Context Manager 不需要一开始就非常复杂，但必须有清晰边界。

4.3.1 Runtime State

```
type RuntimeState = {
  goal: GoalState;
  conversation: Message[];
  memory: MemoryItem[];
  workspace: WorkspaceSnapshot;
  toolResults: ToolResult[];
  plan: PlanState;
  checkpoints: Checkpoint[];
};
```

4.3.2 Context Manager 接口

```
type ContextManager = {
  build(state: RuntimeState, policy: ContextPolicy): ProviderMessage[];
};
```

更完整一点：

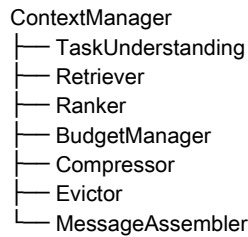
```
type ContextBuildResult = {
  messages: ProviderMessage[];
  usedTokens: number;
  reservedOutputTokens: number;
  selectedItems: ContextItem[];
  evictedItems: ContextItem[];
  compressedItems: ContextItem[];
};
```

```

    trace: ContextBuildTrace;
};

```

4.3.3 内部模块



4.3.4 最小实现策略

第一版可以先实现：

- 最近 N 轮 Conversation ；
- 当前 Goal ；
- 手动选定的文件片段 ；
- 最近关键 Tool Result ；
- 简单 token 估算 ；
- 超限时摘要或截断 ；
- 输出预留 ；
- Context Build Trace。

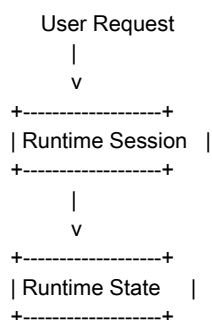
即使简单，也要保留可扩展接口。

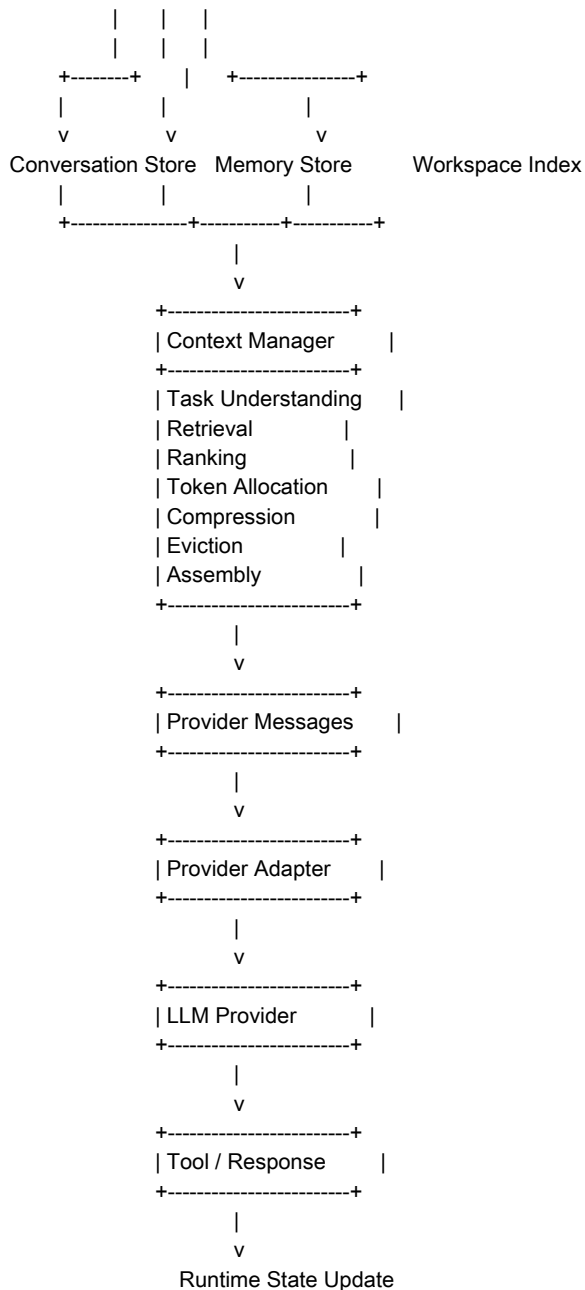
这样后续可以逐步加入：

- embedding retrieval ；
- symbol index ；
- AST index ；
- memory ranking ；
- hierarchical summary ；
- context observability ；
- overflow recovery。

4.4 Day04 总结与收尾

4.4.1 Runtime + Context Engineering 架构图





这张图里最关键的认知是：

- Runtime State 不等于 Context ；
- Context Manager 从多个 Store 和 Index 中选择当前可见信息 ；
- Tool Result 不应该直接进入 LLM ，而应该先进入 Runtime State ；
- Long-running Agent 的连续性来自 State Update、Compression 和 Checkpoint ；
- Provider Adapter 不决定“看什么”，只决定“如何发送给供应商”。

4.4.2 工业级问答补充

问题 1：知识库 Agent 为什么经常不好用？

知识库 Agent 的问题通常不只是“有没有向量库”。

更常见的问题是缺少完整 Context Engineering Pipeline：

- 只有文档切片，没有长期用户 Memory ；

- 只有向量召回，没有 Domain State ；
- 只有相似度排序，没有任务相关 Ranking ；
- 只有原文塞入，没有结构化 Compression ；
- 只有一次问答，没有跨轮 State ；
- 没有对过期、冲突、低价值信息做治理。

所以知识库 Agent 真正需要的是：

Retrieval + Ranking + Memory + Domain State + Context Builder + Governance

问题 2：Memory 会不会越来越大？

会。

Memory 也需要管理。

Memory 不是越多越好，而是需要：

- Memory Consolidation：把零散记忆整合成高质量记忆；
- Memory Ranking：按当前任务相关性排序；
- Memory Eviction：淘汰过期、重复、低价值记忆；
- Memory Refresh：更新旧结论；
- Memory Scope：区分用户级、项目级、任务级记忆。

问题 3：Plan 文档为什么可能失效？

Plan 文档可能是有用 Artifact，但它不是自动可信 Memory。

它可能过期、冲突、缺失因果关系，也可能只是某次任务的中间产物。

Runtime 需要把它纳入 Memory Consolidation 和 Retrieval 流程，才能让它成为可靠的长期上下文来源。

问题 4：Context Builder 像 Compiler 吗？

可以类比成 Compiler。

```
Runtime State -> Source World
Context Policy -> Compile Rule
Context Builder -> Compiler
Provider Message -> Target IR
Provider Adapter -> Backend
LLM Provider -> Execution Target
```

Context Builder 不只是拼字符串，而是在把复杂 Runtime 世界编译成模型可执行的认知输入。

4.4.3 前置问题回收

Day04 Part C 留下的问题

1. Context 超限后如何自动恢复？

通过 Overflow Detection、Ranking、Compression、Eviction、Externalization 和重新构建 Context 完成恢复。

2. Token Budget Manager 如何动态调整？

根据任务类型、当前步骤、输出预留、候选信息 token 成本和历史执行状态动态分配。

3. Context Selection 和 Token Budget Management 的边界是什么？

Selection 决定哪些信息值得进入 Context ; Budget Management 决定它们在有限空间中如何分配。

4. 1M token 为什么仍然不等于无限上下文？

因为成本、延迟、噪声、注意力稀释和输出预留仍然存在。窗口变大只降低约束，不消除治理问题。

本 Part 新增问题

1. Memory 是否也需要 Eviction ？

需要。Memory 是长期状态，但长期状态也会过期、重复、冲突或失去价值。

2. Artifact 如何成为 Memory ？

Artifact 需要经过抽取、验证、整合、标注来源、建立索引和按需检索，才会成为可治理的 Memory。

3. Long-running Agent 的连续性来自哪里？

来自 Runtime State、Summary Memory、Checkpoint、Tool State 和每轮 Context Reconstruction，而不是来自 LLM 自己。

4.4.4 面试视角

1. 什么是 Context Window Management ？

Context Window Management 是 Agent Runtime 对模型可见上下文空间的管理机制。它负责在有限 token 中选择、排序、分配、压缩和淘汰信息，让 LLM 每一轮看到当前任务最需要的信息。

2. Context Window 越大 Agent 越好吗？

不一定。大窗口可以容纳更多信息，但也会带来成本、延迟、噪声和注意力稀释。工业 Agent 的核心不是看更多，而是看对。

3. Runtime State 和 Context 有什么区别？

Runtime State 是 Runtime 的完整世界，Context 是 Runtime 面向当前任务投影给 LLM 的可见世界切片。

4. Context Builder 为什么重要？

因为它决定模型当前看到什么。模型能力再强，如果看到的信息错误、过期、冗余或缺失，推理质量也会下降。

5. Context Builder 和 Prompt Template 有什么区别？

Prompt Template 是静态文本模板；Context Builder 是动态认知管线，会根据 Runtime State、Goal、Policy、Token Budget、Retrieval 和 Compression 生成 Provider Messages。

6. Context Eviction 是删除 Memory 吗？

不是。Context Eviction 是把信息从当前模型输入窗口中移出；Memory 删除是长期存储治理的一部分，两者生命周期不同。

7. Long-running Agent 为什么不会失忆？

不是因为 LLM 自己有持续记忆，而是 Runtime 维护 Runtime State、Summary、Memory、Checkpoint，并在每一轮重新构造 Context。

8. 为什么 Tool Result 不应该直接进入 LLM？

Tool Result 可能很长、噪声多、包含内部细节，也可能与当前任务无关。它应该先进入 Runtime State，再由 Context Builder 选择、压缩和排序。

4.4.5 本 Part 核心认知升级

第一层：从“上下文窗口”到“认知资源”

过去理解：

Context Window = 模型能读多少文本

现在理解：

Context Window = Agent 每一轮推理可用的认知资源

第二层：从“塞更多”到“看对”

过去认为：

模型看到越多越好。

现在理解：

模型需要看到当前任务最相关、最重要、最高密度的信息。

第三层：从“历史聊天”到“Runtime Projection”

过去认为：

Context = 最近聊天记录

现在理解：

Context = Runtime State 经过任务理解、检索、排序、压缩和预算管理后的投影视图

第四层：从“摘要文本”到“状态连续性”

过去认为：

Summary = 把聊天变短

现在理解：

Summary Memory = 保留目标、决策、约束、当前状态和下一步的连续性机制

最终认知

Context Window Management 是 Agent Runtime

的认知资源调度系统。它让模型在有限窗口中持续获得正确的信息密度、任务相关性和状态连续性。

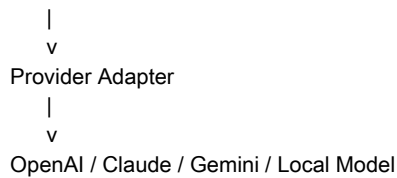
下一节学习计划

下一节继续 Day04 的补充章节：

Day04 Part E：Provider Adapter

这一节补齐 Context Builder 之后的模型供应商适配层：

```
Runtime State
|
v
Context Builder
|
v
Context Object / Provider Messages
```



Provider Adapter 要回答的是：

为什么 Agent Runtime 不应该直接绑定某一家 LLM Provider？

Runtime 内部应该如何定义统一的 LLM Provider Interface？

Context Builder 输出的统一 Context Object 如何转换成 OpenAI、Claude、Gemini 等不同协议？

Streaming、Tool Calling、Usage、Error、Retry 等 provider 差异如何被 Adapter 层屏蔽？

Provider Adapter 和 Context Builder、Tool Runtime、Event System 的边界分别是什么？

完成 Part E 后，再按照学习主线进入：

Day05：Tool Calling

Day05 将承接：

- LLM 如何从 Context 中做出行动决策；
- Tool Call 如何被 Runtime 执行、校验、记录；
- Tool Result 如何回写 Runtime State，并进入下一轮 Context。

写书 TODO

TODO 1：把 Context Engineering 独立成章

建议书稿中单独设置一章：

Context Engineering：Agent 的认知操作系统

这一章可以承接 Prompt Engineering，并说明工业 Agent 真正的关键不再只是写 Prompt，而是管理 Runtime State 到 Context 的投影过程。

TODO 2：加入 Context Window 误区章节

需要明确解释：

- 大 Context 不等于无限记忆；
- Memory 不等于 Conversation；
- Summary 不等于简单压缩；
- RAG 不等于 Context Management；
- Prompt Template 不等于 Context Builder。

TODO 3：加入 Mini Context Manager 实现章节

后续代码实现阶段可以从最小 Context Manager 开始：

- buildContext；
- token estimate；
- recent conversation selection；

- workspace snippet selection ;
- tool result compression ;
- output reserve ;
- build trace。

TODO 4：加入工业案例

可以用 Coding Agent 作为贯穿案例：

- 用户要求修复 bug
- > Runtime 识别任务
- > 检索相关文件
- > 选择错误日志
- > 压缩历史对话
- > 分配 token
- > 组装 Provider Messages
- > LLM 决策调用工具
- > Tool Result 回写 State
- > 下一轮重新构造 Context

写书素材

素材 1：Context 是 Agent 的认知窗口

LLM 每次调用都只活在当前 Context Window 里。Runtime 的责任，是让这个窗口成为当前任务最好的认知切片。

素材 2：Context Builder = Agent Compiler

Context Builder 把 Runtime State 编译成 Provider Messages。它决定信息如何被选择、压缩、排序和呈现。

素材 3：大 Context 不等于强 Agent

真正强的 Agent

不是拥有无限窗口，而是知道什么该进窗口、什么该留在外部、什么该被压缩、什么该被遗忘。

素材 4：Long-running Agent 的连续性来自 Runtime

长任务中的 Agent 不是靠 LLM 自己记住过去，而是靠 Runtime State、Summary、Checkpoint 和 Context Reconstruction 保持连续性。

素材 5：知识库 Agent 的失败往往是 Context Engineering 的失败

很多知识库 Agent

不好用，不是因为缺少向量库，而是因为缺少任务理解、记忆治理、上下文排序、压缩和状态连续性。

本章思考题

为什么 Context Window 不是 Memory？

Conversation、Memory 和 Context Window 的生命周期有什么区别？

为什么 1M token 仍然不能解决所有 Agent 上下文问题？

Token Budget 为什么必须预留 Output Budget？

Context Allocation 如何根据不同任务动态调整？

Priority Ranking 为什么不能只按时间排序？

Context Eviction 和 Memory Eviction 的区别是什么？

Summary Memory 应该保留哪些信息，为什么只保留“聊了什么”不够？

Hierarchical Context 如何帮助 Long-running Agent 保持连续性？

为什么 Plan 文档不自动等于 Memory？

Tool Result 为什么应该先进入 Runtime State，再由 Context Builder 投影？

如果实现一个 Mini Agent Context Manager，第一版最小可用模块应该有哪些？

来源

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a607b7f-6d9c-83e8-ba00-26c6d8c5cad5>

- 本地源记录：`source/day04-part-d-chatgpt-share-source.md`